

Reviewer Pack — Minimal Lattice Point Counting in Algebraic Geometry and Res...

Entry ff6e70f26d3c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 22:42:09 UTC

Minimal Lattice Point Counting in Algebraic Geometry and Resolution Proof Width Inequality

Entry ID: ff6e70f26d3c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 22:42:09 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Lattice Theory) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Boolean satisfiability instance φ with m clauses and n variables, the minimal number of lattice points in the projective space spanned by the vectors associated with φ is linearly correlated with its resolution proof width $w(\varphi)$, such that the ratio of minimal lattice point count to dimension is upper bounded by $O(w(\varphi))$.

Rationale (proposer's reasoning):

Lattice theory has a rich history in studying integer solutions to polynomial equations. By connecting it to resolution complexity, we may uncover new structural insights into the proof complexity of Boolean satisfiability problems, potentially leading to new proof systems or proving techniques.

Taxonomy category: Algebraic Geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c07dd127f48a64a1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all Boolean satisfiability instances φ with m clauses and n variables ($n \leq 40$), the ratio of minimal lattice point count to dimension is less than or equal to $O(w(\varphi))$ AND at least 80% of seeds yield a ratio $\leq O(w(\varphi))$. The conjecture is falsified if any seed produces a ratio $> O(w(\varphi))$ for any instance.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "algebraic geometry" AND "lattice theory" AND "resolution proof complexity" - "projective space" AND "lattice points" AND "Boolean satisfiability" - "resolution proof width" AND "minimal lattice point count" AND "algebraic geometry"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_boolean_instance(n, m):
    variables = [f'x{i}' for i in range(1, n+1)]
    clauses = []
    for _ in range(m):
        clause = random.sample(variables, 2)
        clauses.append(clause)
    return clauses

def compute_lattice_points(clauses):
    vectors = [[0] * (len(clauses) + 1)] + [[c[i] for c in clauses] + [1] for i in range(len(clauses))]
    dimension = len(vectors[0])
    lattice_points = set()
    for v in vectors:
        lattice_points.add(tuple(v))
    return len(lattice_points), dimension

def compute_resolution_width(clauses):
    # Placeholder for actual resolution width computation
    return len(clauses)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 0
    total_ratio = 0.0
    conjecture_holds = True
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
```

```

for _ in range(5): # Sample 5 instances per size
    m = random.randint(n // 2, n)
    clauses = generate_boolean_instance(n, m)
    lattice_points, dimension = compute_lattice_points(clauses)
    width = compute_resolution_width(clauses)
    ratio = Fraction(lattice_points, dimension) if dimension != 0 else Fraction(0, 1)
    total_ratio += ratio
    instances_tested += 1

    if ratio > width:
        conjecture_holds = False
        counterexample = f"n={n}, m={m}, lattice_points={lattice_points}, dimension={dimension}"
        break

mean_ratio = total_ratio / instances_tested
return {
    "metric_name": "Ratio of Lattice Points to Dimension",
    "metric_value": float(mean_ratio),
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unexpected_behavior")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c034f48b.py", line 75, in <module>
    trial_result = run_trial(seed)
    ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c034f48b.py", line 50, in run_trial

```

```
lattice_points, dimension = compute_lattice_points(cclauses)
~~~~~
```

IndexError: list index out of range

Critic verdict: CHALLENGE

skipped: test crashed before producing data

Verdict: INCONCLUSIVE

The test code crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Investigate and fix the crash in the test code to ensure it can run to completion and provide results.

Total LLM calls: 9

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 93330 ms total latency. Provider mix: {'ollama remote': 9}

12. Reproducibility

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ff6e70f26d3c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Replay tarball: `research/replay/ff6e70f26d3c.tar.gz` (if generated)